Introdução ao COBOL e Primeiros Programas

Objetivos da Aula
Ao final desta aula o aluno deverá ser capaz de:

- Compreender a origem e a importância histórica do COBOL
- Instalar e configurar um ambiente para desenvolvimento
- Entender a estrutura básica de um programa COBOL
- Executar seu primeiro programa
- Utilizar os comandos `DISPLAY` e `ACCEPT`
- Desenvolver pequenos programas interativos

1 – Introdução ao COBOL (20 min)

O que é COBOL?

COBOL é a sigla para **Common Business-Oriented Language** (Linguagem Comum Orientada a Negócios).

Criada em **1959**, foi desenvolvida especialmente para **aplicações comerciais e administrativas**.

Características Principais:

Característica	Descrição
Legível	Sintaxe próxima do inglês, facilitando a leitura
Auto documentável	O código descreve o que faz
Portável	Pode rodar em diferentes computadores

Onde é usada hoje?

- 🏦 **Bancos** - sistemas de transações financeiras
- 📊 **Seguradoras** - processamento de apólices
- ✈ **Empresas aéreas** - sistemas de reservas
- 🏛 **Governos** - sistemas previdenciários e fiscais
- 💳 **Cartões de crédito** - processamento de transações

> 📌 ***Dado importante**: Estima-se que **mais de 200 bilhões de linhas** de código COBOL ainda estejam em produção, processando 70% das transações comerciais globais.*

Grace Hopper – A Mãe do COBOL



- **Almirante** da Marinha dos EUA
- **Cientista da computação** visionária
- Acreditava que linguagens deveriam ser **próximas do inglês**
- Desenvolveu o primeiro compilador
- Sua frase famosa: ***"É mais fácil pedir desculpas do que pedir permissão"***

Por que isso importa?
COBOL foi projetado para que **gerentes de negócios** pudessem entender o código, não apenas programadores.

Evolução Histórica

```
1959 ──► COBOL 60 (primeira versão)
1968 ──► COBOL-68 (primeiro padrão ANSI)
1974 ──► COBOL-74 (estruturas de dados)
1985 ──► COBOL-85 (programação estruturada)
2002 ──► COBOL-2002 (orientação a objetos)
2014 ──► COBOL-2014 (último padrão)
```

*> 💡 **Curiosidade**: Programas COBOL escritos na década de 1960 ainda rodam hoje com poucas ou nenhuma modificação!*

2 – Preparando o Ambiente (15 min)

Opção 1 – Ambiente Online (Recomendado para iniciantes)

Não requer instalação e funciona em qualquer navegador.

OneCompiler
🔗 OneCompiler COBOL Online

Vantagens:
- Interface simples
- Execução rápida
- Compartilhamento de código

JDoodle
🔗 JDoodle COBOL Online

Vantagens:
- Permite salvar programas
- Interface limpa
- Suporte a entrada de dados

Opção 2 – Ambiente Desktop (Para instalação local)

GnuCOBOL
🔗 GnuCOBOL Oficial

O compilador COBOL mais usado em ambientes Linux/Windows/Mac.

Instalação no Linux (Ubuntu/Debian):

```
sudo apt-get install gnucobol
```

Instalação no Windows:
- Baixe o instalador do site oficial
- Siga o wizard de instalação

OpenCobolIDE
🔗 OpenCobolIDE no GitHub

IDE com:
- Editor com realce de sintaxe
- Compilação com um clique
- Depuração básica

> 📝 ***Recomendação****: Use o OneCompiler para esta aula, depois instale o GnuCOBOL localmente.*

3 – Estrutura dos Programas COBOL (25 min)

As 4 Divisões do COBOL

Todo programa COBOL tem uma estrutura hierárquica em 4 divisões:

```
IDENTIFICATION DIVISION.  ← Identificação do programa
ENVIRONMENT DIVISION.     ← Ambiente de execução
DATA DIVISION.            ← Dados e variáveis
PROCEDURE DIVISION.       ← Lógica do programa
```

Analogia: Um Programa é como uma Receita de Bolo

Receita	Programa COBOL
Nome da receita	IDENTIFICATION DIVISION
Utensílios necessários	ENVIRONMENT DIVISION
Ingredientes	DATA DIVISION
Modo de preparo	PROCEDURE DIVISION

IDENTIFICATION DIVISION

Para que serve? Identifica o programa.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MEU-PROGRAMA.
AUTHOR. SEU-NOME.          ← Opcional
DATE-WRITTEN. 2026-06-20.  ← Opcional
```

> ⚠ ***IMPORTANTE****: Apenas `PROGRAM-ID` é obrigatório!*

ENVIRONMENT DIVISION

Para que serve? Define o ambiente onde o programa rodará.

Normalmente é usada para:
- Configurar arquivos
- Definir hardware
- Compatibilidade entre sistemas

Nesta disciplina: Será pouco usada inicialmente.

DATA DIVISION

Para que serve? Declara todas as variáveis do programa.

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME    PIC X(30).   ← Variável para texto
01 WS-IDADE   PIC 9(3).    ← Variável para número
```

Explicação: Estudaremos em detalhes na próxima aula.

PROCEDURE DIVISION

Para que serve? Contém todos os comandos executáveis.

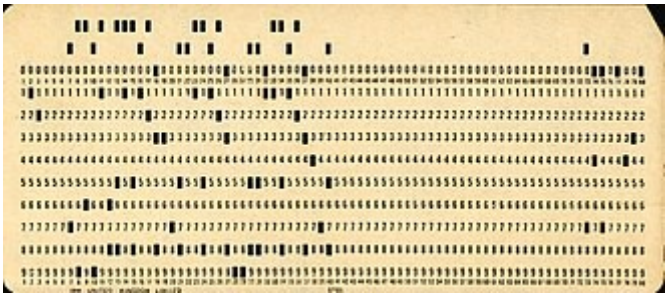
```
PROCEDURE DIVISION.
    DISPLAY "OLÁ MUNDO".   ← Comando para exibir
    STOP RUN.              ← Finaliza o programa
```

> 📌 ***Lembrete**: Toda lógica do programa fica aqui!*

4 – Como os Programas Eram Escritos (15 min)

A Era dos Cartões Perfurados

Antes dos computadores modernos, os programas eram escritos em cartões perfurados.



Como funcionava?
1. O programador digitava o código em uma máquina
2. A máquina perfurava furos no cartão
3. Os cartões eram lidos pelo computador
4. Um erro em um cartão exigia refazer todo o conjunto

Organização das Colunas

Colunas	Uso	Descrição
1-6	Numeração de linha	Identificador do cartão
7	Área de comentário	'*' indica que é comentário
8-11	Área A	Divisões e parágrafos
12-72	Área B	Comandos e código
73-80	Identificação	Uso opcional

Exemplo de código com numeração:

```
000100 IDENTIFICATION DIVISION.
000200 PROGRAM-ID. TESTE.
000300 PROCEDURE DIVISION.
000400     DISPLAY "OLÁ MUNDO".
000500     STOP RUN.
```

Hoje: Formato Livre

O GnuCOBOL aceita o **formato livre** (free format), sem necessidade de numeração ou colunas específicas.

> 💡 ***Curiosidade**: Muitos sistemas legados ainda mantêm a numeração para compatibilidade!*

5 – Primeiro Programa (15 min)

Exemplo 1 – Olá Mundo

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. OLA.

       PROCEDURE DIVISION.
           DISPLAY "OLA MUNDO".
           STOP RUN.
```

Vamos analisar linha por linha:

Linha	O que faz
`IDENTIFICATION DIVISION.`	Inicia a divisão de identificação
`PROGRAM-ID. OLA.`	Nome do programa
`PROCEDURE DIVISION.`	Inicia a parte executável
`DISPLAY "OLA MUNDO".`	Exibe a mensagem
`STOP RUN.`	Finaliza o programa

Executando no OneCompiler:

1. Acesse o OneCompiler
2. Cole o código
3. Clique em "Run"
4. Veja o resultado: `OLA MUNDO`

Resultado esperado:

```
OLA MUNDO
```

Anatomia de um Programa COBOL

```
IDENTIFICATION DIVISION.      ← Divisão 1: Identificação
PROGRAM-ID. OLA.              ← Nome do programa

PROCEDURE DIVISION.           ← Divisão 4: Lógica
    DISPLAY "OLA MUNDO".      ← Comando executável
    STOP RUN.                 ← Comando de fim
```

> 📌 Regras básicas:
> - Cada comando termina com ponto final (`.`)
> - Palavras-chave podem ser maiúsculas ou minúsculas
> - A indentação não é obrigatória, mas recomendada

6 – Comando DISPLAY (10 min)

O que é o DISPLAY?

O comando `DISPLAY` é usado para **exibir informações na tela** do usuário.

Sintaxe:

```
DISPLAY "texto a ser exibido"
```

Exemplos:

```
DISPLAY "BEM-VINDO AO CURSO DE COBOL"
DISPLAY "SISTEMA INICIADO COM SUCESSO"
DISPLAY "AGUARDE O PROCESSAMENTO..."
```

Programa com múltiplos DISPLAYs:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. MENSAGEM.

       PROCEDURE DIVISION.
           DISPLAY "=========================".
           DISPLAY "SISTEMA BANCÁRIO".
           DISPLAY "=========================".
           DISPLAY "PROCESSANDO DADOS...".
           DISPLAY "PROCESSAMENTO CONCLUIDO".
           STOP RUN.
```

Resultado:

```
=========================
SISTEMA BANCÁRIO
=========================
PROCESSANDO DADOS...
PROCESSAMENTO CONCLUIDO
```

> 💡 ***Dica**: Use `DISPLAY` com caracteres especiais para criar interfaces visuais.*

7 – Comando ACCEPT (15 min)

O que é o ACCEPT?

O comando `ACCEPT` permite **receber dados digitados pelo usuário** durante a execução.

Fluxo do programa:

```
1. DISPLAY "DIGITE SEU NOME:"  ← Solicita ao usuário
2. ACCEPT WS-NOME              ← Aguarda entrada do usuário
3. DISPLAY "OLÁ " WS-NOME      ← Exibe o nome recebido
```

Exemplo completo:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. NOME.

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 WS-NOME PIC X(30).   ← Variável para armazenar nome

       PROCEDURE DIVISION.
           DISPLAY "DIGITE SEU NOME:".
           ACCEPT WS-NOME.

           DISPLAY "==================".
           DISPLAY "OLÁ " WS-NOME.
           DISPLAY "SEJA BEM-VINDO!".
           DISPLAY "==================".

           STOP RUN.
```

Resultado da execução:

```
DIGITE SEU NOME:
JOAO    ← Usuário digita e pressiona ENTER
==================
OLÁ JOAO
SEJA BEM-VINDO!
==================
```

Tipos de dados no ACCEPT:

Tipo	Descrição	Exemplo
`PIC X(n)`	Texto (alfanumérico)	`PIC X(30)` = 30 caracteres
`PIC 9(n)`	Números inteiros	`PIC 9(3)` = até 999

*> ⚠ **Importante**: O nome da variável (ex: WS-NOME) deve ser declarado antes de ser usado!*

Exercício Guiado (15 min)

Programa de Saudação Personalizada

Vamos criar juntos um programa que interage com o usuário.

Passo a passo:

Passo 1: Estrutura básica

```
IDENTIFICATION DIVISION.
PROGRAM-ID. SAUDACAO.
```

Passo 2: Declarar variáveis

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME    PIC X(30).
01 WS-CIDADE  PIC X(30).
```

Passo 3: Lógica do programa

```
PROCEDURE DIVISION.
    DISPLAY "DIGITE SEU NOME:".
    ACCEPT WS-NOME.

    DISPLAY "DIGITE SUA CIDADE:".
    ACCEPT WS-CIDADE.

    DISPLAY "=========================".
    DISPLAY "OLÁ " WS-NOME.
    DISPLAY "SEJA BEM-VINDO A " WS-CIDADE.
    DISPLAY "=========================".

    STOP RUN.
```

Programa completo:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. SAUDACAO.

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 WS-NOME    PIC X(30).
       01 WS-CIDADE  PIC X(30).

       PROCEDURE DIVISION.
           DISPLAY "DIGITE SEU NOME:".
           ACCEPT WS-NOME.

           DISPLAY "DIGITE SUA CIDADE:".
           ACCEPT WS-CIDADE.

           DISPLAY "=========================".
           DISPLAY "OLÁ " WS-NOME.
           DISPLAY "SEJA BEM-VINDO A " WS-CIDADE.
           DISPLAY "=========================".

           STOP RUN.
```

Teste de execução:

Entrada	Saída
Nome: JOAO	`OLÁ JOAO`
Cidade: BARUERI	`SEJA BEM-VINDO A BARUERI`

Resultado final:

```
DIGITE SEU NOME:
JOAO
DIGITE SUA CIDADE:
BARUERI
=========================
OLÁ JOAO
SEJA BEM-VINDO A BARUERI
=========================
```

Atividade Prática da Aula (30 min)

🎯 **Sistema de Apresentação Pessoal**

Objetivo:
Criar um programa que colete dados pessoais do usuário e exiba uma ficha organizada.

Requisitos técnicos:

Requisito	Descrição
✅	Utilizar `DISPLAY` para mensagens
✅	Utilizar `ACCEPT` para entrada de dados
✅	Declarar variáveis na `WORKING-STORAGE`
✅	Exibir saída formatada com separadores
✅	Criar títulos e bordas visuais

Dados a serem solicitados:

1. 👄 **Nome completo**
2. 🌆 **Cidade**
3. 🎓 **Curso**
4. 🔢 **Idade**

Formato da saída:

```
========================================
        FICHA DO ALUNO
========================================

Nome  : José da Silva
Cidade: Barueri
Curso : Análise e Desenvolvimento de Sistemas
Idade : 35 anos

========================================
        CADASTRO REALIZADO
========================================
```

📝 Passo a Passo da Construção

Estrutura recomendada:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. FICHA-ALUNO.

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 WS-NOME    PIC X(40).
       01 WS-CIDADE  PIC X(30).
       01 WS-CURSO   PIC X(50).
       01 WS-IDADE   PIC 9(3).

       PROCEDURE DIVISION.
           DISPLAY "======================================".
           DISPLAY "      SISTEMA DE CADASTRO DE ALUNO".
           DISPLAY "======================================".

           DISPLAY " ".
           DISPLAY "DIGITE SEU NOME:".
           ACCEPT WS-NOME.

           DISPLAY "DIGITE SUA CIDADE:".
           ACCEPT WS-CIDADE.

           DISPLAY "DIGITE SEU CURSO:".
           ACCEPT WS-CURSO.

           DISPLAY "DIGITE SUA IDADE:".
           ACCEPT WS-IDADE.

           DISPLAY " ".
           DISPLAY "======================================".
           DISPLAY "        FICHA DO ALUNO".
           DISPLAY "======================================".
           DISPLAY " ".
           DISPLAY "Nome  : " WS-NOME.
           DISPLAY "Cidade: " WS-CIDADE.
           DISPLAY "Curso : " WS-CURSO.
           DISPLAY "Idade : " WS-IDADE " ANOS".
           DISPLAY " ".
           DISPLAY "======================================".
           DISPLAY "      CADASTRO REALIZADO COM SUCESSO".
           DISPLAY "======================================".

           STOP RUN.
```

🔍 Pontos de atenção:

Linha	Explicação
`WS-NOME PIC X(40)`	Reserva 40 caracteres para o nome
`WS-IDADE PIC 9(3)`	Reserva 3 dígitos para idade (0-999)
`DISPLAY " "`	Linha em branco para melhor visualização
`DISPLAY WS-IDADE " ANOS"`	Concatena idade com texto

🎨 Variações para praticar

Versão 1 - Com borda dupla:

```
DISPLAY "========================================".
DISPLAY "||        FICHA DO ALUNO              ||".
DISPLAY "========================================".
```

Versão 2 - Com linha pontilhada:

```
DISPLAY "----------------------------------------".
```

Versão 3 - Com emojis (se suportado):

```
DISPLAY "📋 FICHA DO ALUNO".
```

> 💡 ***Desafio:*** *Tente criar diferentes estilos de borda!*

Desafio Extra (Opcional)

🏆 Sistema Completo de Cadastro

Adicione mais informações:

1. 💼 **Profissão**
2. 📧 **E-mail**
3. 📱 **Telefone**
4. 📅 **Data de Nascimento**

Saída esperada:

```
============================================================
                    FICHA COMPLETA DO ALUNO
============================================================

DADOS PESSOAIS
------------------------------------------
Nome       : José da Silva
Idade      : 35 anos
Cidade     : Barueri
Profissão  : Analista de Sistemas

DADOS ACADÊMICOS
------------------------------------------
Curso      : Análise e Desenvolvimento de Sistemas

CONTATO
------------------------------------------
E-mail     : jose@email.com
Telefone   : (11) 99999-9999

============================================================
```

Desafios adicionais:

- **Nível 1:** Adicionar uma nova variável
- **Nível 2:** Criar diferentes seções com títulos
- **Nível 3:** Usar diferentes tipos de borda
- **Nível 4:** Incluir validação de dados (aceitar apenas números para idade)

Material Complementar

📚 **Documentação Recomendada**

Para iniciantes:

1. **GnuCOBOL Programmer's Guide** - Tutorial passo a passo
   - 🔗 Link
2. **COBOL Quick Reference** - Resumo da linguagem
   - 🔗 Link
3. **Manual do GnuCOBOL** - Referência completa
   - 🔗 Link

Canais recomendados:

- 📺 **YouTube**: "COBOL Programming Tutorials"
- 📝 **Blog**: "COBOL Cafe" - dicas e truques
- 💬 **Fórum**: Stack Overflow - tag #COBOL